Lab Report: Lab 2 - Layered Architecture Design (OmniDash)

Contents

1. Cover Page

- **Title:** Logical View Design and Layered Architecture for the OmniDash Productivity Platform
- **Course Info:** Software Architecture
- **Student Details:** Nguyễn Văn Trường (23010371)

2. Abstract/Summary

Following the requirements elicitation in Lab 1, this lab focuses on the **Logical View** of the OmniDash platform. The objective is to design a robust **Modular Layered Architecture** that effectively addresses our Architecturally Significant Requirements (ASRs), specifically regarding **Extensibility** and **Integrability**.

In this session, we formally define four architectural layers (Presentation, Business Logic, Persistence, and Data), identify core software components for the "Workspace Management" module, and establish strict dependency rules.

This logical blueprint ensures that the system can handle complex interactions between the Web App, AI Service, and Browser Extension while maintaining a clean separation of concerns.

3. Lab Specific Section: II. Layered Architecture Design

3.1 Defining Layers and Responsibilities

The OmniDash system follows a strict downward dependency rule, ensuring that higher layers remain decoupled from the technical implementation details of lower layers.

Layer	Purpose / Responsibility	Key Artifacts
1. Presentation Layer	Handles incoming HTTP requests from the Next.js UI or Chrome Extension. Manages routing, input validation (DTOs), and session-based authentication.	WorkspaceController, TaskController, AuthController
2. Business Logic Layer	The core logic of the system. Implements workspace orchestration, AI task decomposition rules, and context-switching logic.	WorkspaceService, AIService, TaskService
3. Persistence Layer	Abstracts data access logic. Uses Prisma ORM to map business objects to the PostgreSQL schema and execute CRUD operations.	WorkspaceRepository, TaskRepository
4. Data Layer	The physical storage tier where user data, workspace configurations, tasks, and logs reside.	**PostgreSQL Database**

3.2 Component Identification (Feature: Workspace Activation)

To illustrate the architecture, we decompose the **"Activate Workspace Combo"** feature into specific components across the layers:

Typical Execution Flow:

1. **Presentation Layer:** The WorkspaceController receives a request containing a workspaceId. It validates the session and calls the Service layer.
2. **Business Logic Layer:** The WorkspaceService verifies ownership, retrieves the resource list, and prepares the command for the Browser Extension.
3. **Persistence Layer:** The WorkspaceRepository uses the Prisma client to execute a query against the workspaces and resources tables.
4. **Data Layer:** PostgreSQL returns the requested records to the repository.

Interface Definitions:

- **IWorkspaceService (for Layer 1):** public Workspace getWorkspaceDetails(String workspaceId)
- **IWorkspaceRepository (for Layer 2):** public WorkspaceEntity findById(String workspaceId)

3.3 Component Diagram Modeling

The logical structure is modeled using a UML Component Diagram, enforcing the "Strict Layering" rule.

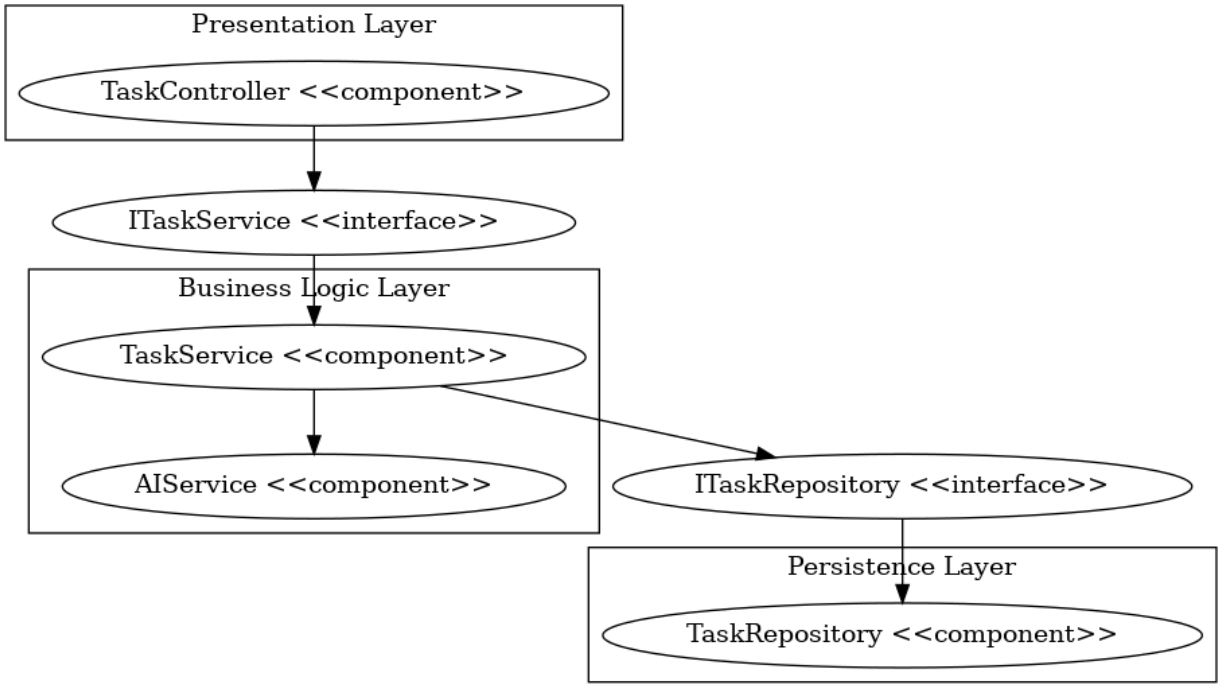
- **Provided Interfaces (Lollipops):** IWorkspaceService and ITaskRepository.
- **Required Interfaces (Sockets):** WorkspaceController requiring IWorkspaceService.
- **Dependencies:** All dashed arrows point strictly downwards (Layer 1 → Layer 2 → Layer 3), preventing circular dependencies.

4. Architectural Analysis (Preparation for Implementation)

The proposed Layered Architecture directly addresses the ASRs identified in the previous phase:

- **ASR-1 (Integrability) & Presentation Layer:** By isolating request handling, we provide a unified API endpoint for both the Web App and the Browser Extension, ensuring consistent behavior across platforms.

- **ASR-2 (Extensibility) & Service Layer:** The modularity of the Service layer allows us to add new AI-driven features or productivity widgets as isolated services without affecting existing authentication or task logic.
- **ASR-3 (Performance) & Repository Layer:** Utilizing Prisma ORM within the Persistence layer enables optimized database queries and connection pooling, critical for meeting the <1.5s load time target (NFR-01).



5. Conclusion & Reflection

5.1 Conclusion

Lab 2 has successfully translated the functional scope of OmniDash into a concrete **Logical View**. By defining a four-layer architecture, we have created a clear boundary for code organization and dependency management. This structural clarity is essential for our Full-stack JavaScript environment, where maintaining a clean separation between UI components and complex business logic is often difficult.

5.2 Reflection

Designing the logical architecture highlighted several critical points:

- **Interface-First Thinking:** Defining method signatures before implementation forces a focus on *what* a component does rather than *how* nó thực hiện.
- **Strict Layering Benefits:** Realizing that the Controller should never communicate directly with the Database protects system integrity and allows for easier testing of business logic in isolation.
- **Architecture Documentation:** Using UML diagrams helped identify potential "architectural leaks" early, ensuring a cleaner codebase during the development cycle